



Lab FC

Guia de Desplegament a Produccio

Instruccions per a l'equip de sistemes
Desplegament en Ubuntu Server amb Apache

Versio	2.0
Data	Marc 2026
Entorn	Ubuntu Server 24.04 + Apache + Gunicorn + PostgreSQL
Repositori	https://github.com/ohijazo/Lab-FA-PostgreSQL
Destinatari	Equip de Sistemes

1. Requisits de la Maquina Virtual

L'aplicacio Lab FC es lleugera i no requereix grans recursos. Es pot desplegar en una maquina virtual amb **Ubuntu Server 24.04 LTS**. Ubuntu es gratuït, estable i ideal per a servidors.

Especificacions recomanades

Recurs	Minim	Recomanat
CPU	2 cores	4 cores
RAM	2 GB	4 GB
Disc	20 GB	50 GB SSD
Sistema Operatiu	Ubuntu Server 22.04 LTS	Ubuntu Server 24.04 LTS
Xarxa	Connectivitat LAN	IP fixa a la xarxa local

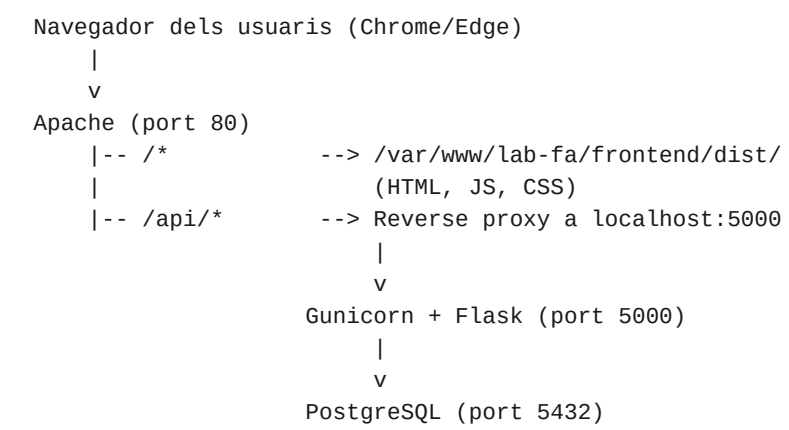
NOTA: Ubuntu Server sense entorn grafic consumeix molta menys RAM que Windows (~300 MB vs ~3 GB). Amb 4 GB de RAM la maquina anira molt sobrada. Es l'opcio mes eficient en recursos.

Justificacio dels recursos

Component	Consum estimat	Observacions
Sistema Operatiu (Ubuntu)	~300 MB RAM	Sense entorn grafic, molt lleuger
Apache	~50 MB RAM	Servidor web robust i provat
Backend (Gunicorn + Flask)	~150 MB RAM	4 workers, servidor WSGI natiu Linux
PostgreSQL	~100-200 MB RAM	Base de dades amb pocs registres
Aplicacio (codi + frontend)	< 100 MB disc	Fitxers estatics compilats
Base de dades (10.000 reg.)	< 50 MB disc	Creix segons volum d'analisi

2. Visio General de l'Arquitectura

Lab FC es una aplicacio web composta per dues parts: un **frontend** (interficie web) i un **backend** (API REST). **Apache** serveix els fitxers estatics del frontend i redirigeix les peticions **/api/*** al backend Python via reverse proxy. **Gunicorn** es el servidor WSGI natiu de Linux (equivalent a Waitress en Windows).



Components necessaris

Component	Versio	Funcio
Ubuntu Server	24.04 LTS	Sistema operatiu
Python	3.12+ (inclos a Ubuntu)	Executa el backend (API REST)
Node.js	18+ (recomanat 20 LTS)	Compilar el frontend (un sol cop)
PostgreSQL	16 (inclos a Ubuntu 24.04)	Base de dades
Apache	2.4 (inclos a Ubuntu)	Servidor web + reverse proxy
Gunicorn	22+	Servidor WSGI (instal.lat amb pip)
Git	2.40+	Descarregar i actualitzar el codi

AVANTATGE: La majoria de components (Python, PostgreSQL, Apache, Git) ja venen inclosos o s'instal·len fàcilment amb apt. No cal descarregar instal·ladors manualment.

3. Instal·lacio de Paquets

Pas 3.1 - Actualitzar el sistema

```
sudo apt update && sudo apt upgrade -y
```

Pas 3.2 - Instal·lar tots els paquets necessaris

```
# Python, pip, venv
sudo apt install -y python3 python3-pip python3-venv

# PostgreSQL
sudo apt install -y postgresql postgresql-contrib

# Apache i modul proxy
sudo apt install -y apache2

# Node.js (per compilar el frontend)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Git
sudo apt install -y git
```

Pas 3.3 - Verificar instal·lacions

```
python3 --version    # 3.12+
node --version        # 20+
psql --version        # 16+
apache2 -v            # 2.4+
git --version         # 2.40+
```

4. Configurar PostgreSQL

Pas 4.1 - Crear la base de dades i l'usuari

Connectar a PostgreSQL com a superusuari:

```
sudo -u postgres psql
```

Executar les següents comandes SQL:

```
CREATE DATABASE labfa;  
CREATE USER labfa_user WITH PASSWORD 'UnaContrasenyaSegura123';  
GRANT ALL PRIVILEGES ON DATABASE labfa TO labfa_user;
```

```
-- Connectar a la BD labfa i donar permisos a l'schema:  
\c labfa  
GRANT ALL ON SCHEMA public TO labfa_user;
```

```
-- Sortir:  
\q
```

IMPORTANT: Canviar 'UnaContrasenyaSegura123' per una contrasenya segura real. Anotar-la perquè es necessitarà al fitxer .env del backend.

5. Descarregar i Configurar l'Aplicacio

Pas 5.1 - Clonar el repositori

```
sudo mkdir -p /var/www
cd /var/www
sudo git clone https://github.com/ohijazo/Lab-FA-PostgreSQL.git lab-fa
sudo chown -R $USER:$USER /var/www/lab-fa
```

Pas 5.2 - Crear l'entorn virtual Python

```
cd /var/www/lab-fa/backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install gunicorn
```

Gunicorn es el servidor WSGI de produccio per Linux. Es mes eficient que Waitress (usat a Windows) i es l'estandard en entorns Linux.

Pas 5.3 - Crear el fitxer de configuracio .env

Copiar **.env.example** a **.env** i editar:

```
cd /var/www/lab-fa/backend
cp .env.example .env
nano .env
```

Contingut del fitxer .env:

```
DATABASE_URL=postgresql://labfa_user:ContrasenyaReal@localhost:5432/labfa
SECRET_KEY=clau-aleatoria-de-minim-32-caracters
ADMIN_EMAIL=admin@agrienergia.com
ADMIN_PASSWORD=ContrasenyaAdminSegura
CORS_ORIGINS=http://labfc.agrienergia.local
FLASK_DEBUG=0
```

Variable	Descripcio
DATABASE_URL	Connexio PostgreSQL: postgresql://usuari:password@host:port/bd
SECRET_KEY	Clau secreta per a sessions (generar amb: python3 -c "import secrets; print(secrets.token_hex(32))")
ADMIN_EMAIL	Email de l'usuari administrador inicial
ADMIN_PASSWORD	Contrasenya de l'usuari administrador inicial
CORS_ORIGINS	URL d'accés dels usuaris (IP o domini del servidor)
FLASK_DEBUG	Posar sempre a 0 en produccio

IMPORTANT: CORS_ORIGINS ha de coincidir exactament amb la URL que els usuaris escriuen al navegador. Si accedeixen per http://192.168.1.50, posar exactament això. Si es fa servir un nom DNS com http://labfc.agrienergia.local, posar el nom DNS.

Pas 5.4 - Inicialitzar la base de dades

```
cd /var/www/lab-fa/backend
source venv/bin/activate
```

```
# Crear totes les taules
flask db upgrade

# Crear l'usuari administrador inicial
flask seed-admin
```

Despres d'executar **seed-admin**, es pot entrar a l'aplicacio amb l'email i contrasenya definits a ADMIN_EMAIL i ADMIN_PASSWORD.

Pas 5.5 - Verificar el backend manualment

```
cd /var/www/lab-fa/backend
source venv/bin/activate
gunicorn -w 4 -b 0.0.0.0:5000 'app:create_app()'

# Des d'un altre terminal:
curl http://localhost:5000/api/tipus
# Hauria de retornar un JSON (llista buida [])
# Ctrl+C per aturar
```

6. Compilar el Frontend

Pas 6.1 - Instal·lar dependències i compilar

```
cd /var/www/lab-fa/frontend
```

```
# Instal·lar dependències  
npm install
```

```
# Compilar per producció  
npm run build
```

Això genera la carpeta **frontend/dist/** amb els fitxers estàtics (HTML, JS, CSS). Apache servirà aquests fitxers directament. Un cop compilat, Node.js ja no es necessita fins a la propera actualització.

NOTA: npm run build només cal executar-lo un cop, o cada vegada que s'actualitzi el codi font. Node.js no ha d'estar en execució permanent.

7. Backend com a Servei (systemd)

Per garantir que el backend s'arrenqui automaticament amb la maquina i es reinicii si cau, es registra com a servei **systemd** (el gestor de serveis natiu d'Ubuntu).

Pas 7.1 - Crear el fitxer de servei

Crear el fitxer **/etc/systemd/system/labfc.service**:

```
sudo nano /etc/systemd/system/labfc.service
```

Contingut:

```
[Unit]
Description=Lab FC - Backend API
After=network.target postgresql.service

[Service]
User=www-data
Group=www-data
WorkingDirectory=/var/www/lab-fa/backend
EnvironmentFile=/var/www/lab-fa/backend/.env
ExecStart=/var/www/lab-fa/backend/venv/bin/gunicorn \
    -w 4 \
    -b 127.0.0.1:5000 \
    --access-logfile /var/log/labfc/access.log \
    --error-logfile /var/log/labfc/error.log \
    'app:create_app()'
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Pas 7.2 - Preparar permisos i logs

```
# Crear carpeta de logs
sudo mkdir -p /var/log/labfc
sudo chown www-data:www-data /var/log/labfc

# Donar permisos a www-data sobre l'aplicacio
sudo chown -R www-data:www-data /var/www/lab-fa
```

Pas 7.3 - Activar i arrencar el servei

```
# Recarregar configuracio de systemd
sudo systemctl daemon-reload

# Activar arrencada automatica
sudo systemctl enable labfc

# Arrencar el servei
sudo systemctl start labfc

# Verificar l'estat
sudo systemctl status labfc
# Hauria de dir: active (running)
```

Comandes de gestio del servei

Comanda	Accio
sudo systemctl start labfc	Arrencar el servei
sudo systemctl stop labfc	Aturar el servei
sudo systemctl restart labfc	Reiniciar el servei
sudo systemctl status labfc	Veure l'estat
sudo journalctl -u labfc -f	Veure logs en temps real
sudo systemctl disable labfc	Desactivar arrencada automatica

8. Configurar Apache

Apache fa dues funcions: servir els fitxers estatics del frontend i redirigir les peticions **/api/*** al backend Gunicorn via reverse proxy.

Pas 8.1 - Activar moduls necessaris

```
sudo a2enmod proxy proxy_http rewrite
sudo systemctl restart apache2
```

Pas 8.2 - Crear el VirtualHost

Crear el fitxer **/etc/apache2/sites-available/labfc.conf**:

```
sudo nano /etc/apache2/sites-available/labfc.conf
```

Contingut:

```
<VirtualHost *:80>
    ServerName labfc.agrienergia.local
    # Si tambe es vol acces per IP, descomentar:
    # ServerAlias 192.168.1.50

    # Frontend: fitxers estatics compilats
    DocumentRoot /var/www/lab-fa/frontend/dist

    <Directory /var/www/lab-fa/frontend/dist>
        Options -Indexes +FollowSymLinks
        AllowOverride All
        Require all granted

        # SPA: redirigir totes les rutes a index.html
        RewriteEngine On
        RewriteBase /
        RewriteCond %{REQUEST_FILENAME} !-f
        RewriteCond %{REQUEST_FILENAME} !-d
        RewriteCond %{REQUEST_URI} !^/api/
        RewriteRule ^ /index.html [L]
    </Directory>

    # Proxy API -> backend Gunicorn
    ProxyPreserveHost On
    ProxyPass /api/ http://127.0.0.1:5000/api/
    ProxyPassReverse /api/ http://127.0.0.1:5000/api/

    # Logs
    ErrorLog ${APACHE_LOG_DIR}/labfc-error.log
    CustomLog ${APACHE_LOG_DIR}/labfc-access.log combined
</VirtualHost>
```

Pas 8.3 - Activar el site i desactivar el default

```
# Activar el site de Lab FC
sudo a2ensite labfc.conf

# Desactivar el site per defecte d'Apache
sudo a2dissite 000-default.conf

# Verificar la configuracio
sudo apache2ctl configtest
```

```
# Hauria de dir: Syntax OK
```

```
# Reiniciar Apache
```

```
sudo systemctl restart apache2
```

NOTA: La regla RewriteCond exclou les peticions /api/ del rewrite SPA, perquè aquestes les gestiona el ProxyPass cap al backend. La resta de rutes van a index.html perquè React gestioni el routing.

9. Configuracio de Xarxa

Firewall (UFW)

Configurar el firewall d'Ubuntu per permetre acces HTTP:

```
# Permetre SSH (important! no perdre acces remot)
sudo ufw allow ssh

# Permetre HTTP
sudo ufw allow 80/tcp

# Activar el firewall
sudo ufw enable

# Verificar regles
sudo ufw status
```

No cal obrir el port 5000 ni el 5432 - tant Unicorn com PostgreSQL nomes son accessibles des de la propia maquina (127.0.0.1).

DNS intern (opcional pero recomanat)

Crear una entrada DNS al servidor DNS de l'empresa perque els usuaris puguin accedir per nom:

```
Tipus:    A
Nom:      labfc.agrienergia.local
Valor:    IP-DE-LA-MAQUINA (ex: 192.168.1.50)
```

Si no es configura DNS, els usuaris accediran per IP directament (<http://192.168.1.50>).

IP fixa (recomanat)

Configurar IP fixa editant el fitxer de Netplan:

```
sudo nano /etc/netplan/00-installer-config.yaml
```

Exemple de configuracio:

```
network:
  version: 2
  ethernets:
    ens18:                # Nom de la interficie (pot variar)
      addresses:
        - 192.168.1.50/24
      gateway4: 192.168.1.1
      nameservers:
        addresses:
          - 192.168.1.1
          - 8.8.8.8
```

```
sudo netplan apply
```

10. Verificacio Final

Checklist de verificacio

#	Verificacio	Com comprovar-ho
1	PostgreSQL actiu	<code>sudo systemctl status postgresql</code>
2	Backend actiu	<code>sudo systemctl status labfc</code>
3	Apache actiu	<code>sudo systemctl status apache2</code>
4	API respon	<code>curl http://localhost/api/tipus</code> retorna []
5	Frontend accessible	Obrir <code>http://labfc.agrienergia.local</code> al navegador
6	Login admin	Entrar amb ADMIN_EMAIL / ADMIN_PASSWORD
7	Acces des d'altre PC	Obrir <code>http://IP-MAQUINA</code> des d'un PC de la xarxa
8	Auto-start	Reiniciar la maquina: <code>sudo reboot</code>

Prova completa

Des d'un PC de la xarxa local, obrir el navegador (Chrome o Edge) i anar a:

`http://labfc.agrienergia.local`

`http://IP-DE-LA-MAQUINA`

Hauria d'apareixer la pantalla de login. Entrar amb les credencials d'admin.

11. Backups i Manteniment

Backup de la base de dades

Backup manual:

```
sudo -u postgres pg_dump -F c labfa > /backups/labfa_$(date +%Y%m%d).dump

# Restaurar:
sudo -u postgres pg_restore -d labfa /backups/labfa_20260324.dump
```

Backup automatic amb cron

Crear un script i programar-lo amb cron:

```
# Crear carpeta de backups
sudo mkdir -p /backups
sudo chown postgres:postgres /backups

# Crear script de backup
sudo nano /usr/local/bin/backup-labfc.sh
```

Contingut del script:

```
#!/bin/bash
BACKUP_DIR=/backups
DATE=$(date +%Y%m%d)
pg_dump -U postgres -F c labfa > $BACKUP_DIR/labfa_$DATE.dump

# Eliminar backups de mes de 30 dies
find $BACKUP_DIR -name 'labfa_*.dump' -mtime +30 -delete

# Donar permisos d'execucio
sudo chmod +x /usr/local/bin/backup-labfc.sh

# Programar backup diari a les 2:00 AM
sudo crontab -e
# Afegir la linia:
# 0 2 * * * /usr/local/bin/backup-labfc.sh
```

Actualitzar l'aplicacio

Quan es lliuri una nova versio del codi:

```
# 1. Descarregar la nova versio
cd /var/www/lab-fa
sudo git pull

# 2. Actualitzar dependencies backend
cd backend
source venv/bin/activate
pip install -r requirements.txt
flask db upgrade

# 3. Recompilar frontend
cd ../frontend
npm install
npm run build

# 4. Restaurar permisos
sudo chown -R www-data:www-data /var/www/lab-fa
```

```
# 5. Reiniciar el backend
sudo systemctl restart labfc
```

Logs

Log	Ubicacio
Backend (Gunicorn) acces	/var/log/labfc/access.log
Backend (Gunicorn) errors	/var/log/labfc/error.log
Apache acces	/var/log/apache2/labfc-access.log
Apache errors	/var/log/apache2/labfc-error.log
PostgreSQL	/var/log/postgresql/
Servei systemd	sudo journalctl -u labfc

12. Resum

Recursos de la maquina virtual

Recurs	Recomanat
CPU	4 cores
RAM	4 GB (suficient gracies a Linux)
Disc	50 GB SSD
SO	Ubuntu Server 24.04 LTS
Ports	22 (SSH, administracio) - 80 (HTTP, acces usuaris)
Navegadors suportats	Chrome 90+, Edge 90+ (escriptori i mobil)
Usuaris simultaneos	Dissenyat per a equip de laboratori (5-20 usuaris)

Resum de passos

Pas	Accio	Temps estimat
1	Crear maquina virtual Ubuntu Server 24.04	15 min
2	Instal.lar paquets (apt install)	10 min
3	Crear BD i usuari PostgreSQL	5 min
4	Clonar repositori i configurar backend	10 min
5	Compilar frontend (npm install + build)	5 min
6	Crear servei systemd per Gunicorn	5 min
7	Configurar Apache (VirtualHost + proxy)	10 min
8	Configurar firewall, DNS i IP fixa	10 min
9	Verificacio final	5 min

Temps total estimat: ~75 minuts

Comparativa d'opcions de desplegament

Aspecte	Windows Server + IIS	Windows 11 + Waitress	Ubuntu + Apache
Llicencia SO	De pagament	De pagament	Gratuit
RAM necessaria	8 GB	8 GB	4 GB
Servidor web	IIS (complex)	Waitress (simple)	Apache (robust)
Gestor serveis	NSSM (extern)	NSSM (extern)	systemd (natiu)
Complexitat	Alta	Baixa	Mitjana
Estabilitat	Bona	Bona	Excel.lent
Manteniment	GUI (IIS Manager)	CLI / GUI	CLI (SSH)
Backups auto.	Task Scheduler	Task Scheduler	cron (natiu)